

# Monitoring Talend with Datadog

Log collection & forwarding, AI-driven incident response, and a hands-on PoC runbook

Solution brief & lab guide · Prepared May 2026

**Contents**

1. Talend → Datadog log integration (architecture & methods)
2. AI-driven incident investigation & response (Bits AI SRE)
3. Reducing MTTR & on-call effort
4. Developer support & automation (Bits AI Dev Agent)
5. Security monitoring & triage (Bits AI Security Analyst)
6. Licensing & feature access
7. PoC approach & integrations (Slack, ServiceNow, PagerDuty)
8. Automation & proactive monitoring (Bits AI + Watchdog)
9. Kubernetes monitoring setup
10. Unified observability dashboards
11. Hands-on labs (1–6)
12. Pre-flight checklist

## 1. Talend → Datadog log integration

There is no native Talend connector, but that is not a blocker: Talend logging runs on Apache Log4j, so you treat it like any JVM application. Talend Studio embeds Log4j for runtime logging, loggers can be enabled or disabled per component, and the standard levels are Trace, Debug, Info, Warn, Error, and Fatal. The integration problem therefore reduces to getting Log4j output into Datadog cleanly. There are three viable patterns, all converging on the Datadog Agent and platform.

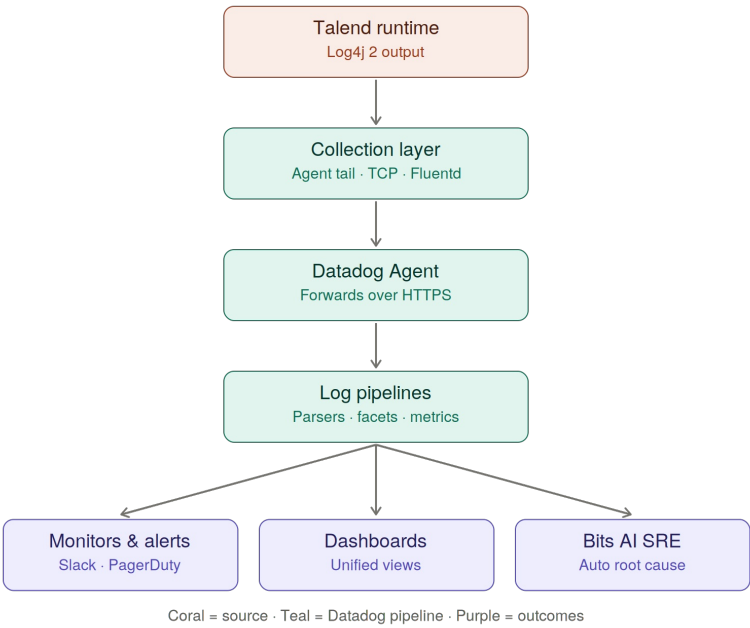


Figure 1 — Talend Log4j output flows through a collection layer and the Agent into Datadog pipelines, then fans out to alerting, dashboards, and the AI layer.

### Pattern A — Datadog Agent tailing log files (recommended)

When you control the host where Talend Job Servers or Remote Engines run, this is the most robust, lowest-effort approach. Log collection is off by default; you enable it in `datadog.yaml` and then point the Agent at Talend's log files via a custom source. Multi-line aggregation is essential here so that a Java stack trace from a `tDie` or exception is captured as one event rather than dozens of fragments. Use unified service tagging (`source:talend`, `service:talend-etl`) so logs correlate with metrics and traces.

### Pattern B — Log4j/Logback appender to the local Agent

Instead of writing to disk, stream Talend's output to the Agent over a TCP socket. Note the key constraint: Datadog does not accept logs sent directly over TCP to its intake — the appender targets the *local Agent*, which then forwards over HTTPS with enrichment. The strong recommendation is to emit structured JSON: Talend's Log4j 2 MDC carries arbitrary key-value pairs, so pushing job name, execution ID, status, and row counts into the MDC means Datadog parses them into facets automatically with no grok work.

**Security note:** Given the Log4Shell history (CVE-2021-44228), confirm you are on a patched Log4j 2 build. Talend also advises disabling Log4j loggers in components when running Spark Batch/Streaming jobs on certain distributions (Dataproc 2.x, Azure Synapse, HD Insight 4.0).

## Pattern C — Talend Cloud / Remote Engine & the HTTP API

For Talend Cloud, jobs execute on a Remote Engine running on your own infrastructure, so Pattern A still applies — install the Agent on the Remote Engine host. Alternatively, any process that can POST over HTTPS can use Datadog's Log HTTP API (1 MB max per request; keep individual logs under ~25 KB). If you already run a logging pipeline, an open-source shipper such as Rsyslog, Syslog-ng, NXLog, Fluentd, or Logstash can forward to Datadog or to a local Agent — and Observability Pipelines is the option when you want to preprocess log data in your environment before shipping.

### Once logs land in Datadog

- **Pipelines & parsers** extract job name, status, duration, and row counts into facets (trivial with JSON/MDC; otherwise a grok parser).
- **Log-based metrics** turn matching logs into a metric such as `talend.job.failures`, so you alert on rates and trends rather than single lines.
- **Log monitors** alert on ERROR/FATAL volume, and a log-absence monitor catches schedules that silently never ran.
- **Dashboards** combine job success rate, runtime, error counts, and host/JVM metrics from the same Agent.
- **Routing** to Slack, PagerDuty, and ServiceNow via monitor notifications.

## 2. AI-driven incident investigation & response (Bits AI SRE)

---

Bits AI SRE, launched December 2025, is Datadog's flagship AI agent: it is aware of telemetry, architecture, and organizational context, and surfaces an actionable root cause within minutes of an alert. When an alert fires it analyzes runbooks and telemetry, separates signal from noise, validates its own findings, and delivers a conclusion to collaboration tools — often before the on-call engineer logs in. Mechanically it runs a continuous loop of observation, reasoning, and action: forming hypotheses, querying telemetry to confirm or reject them, and chaining steps until it converges on the most likely cause. The March 2026 update broadened it to correlate logs, metrics, and infrastructure state across multi-dependency alerts, and added third-party integrations (GitHub, ServiceNow, Grafana, Splunk, Dynatrace, Sentry), a tunable `bits.md` knowledge file, and an API.

## 3. Reducing MTTR & on-call effort

---

This is the core value of Bits AI SRE. Datadog reports it pinpoints root causes in minutes and helps teams restore service dramatically faster, with customer references citing large MTTR reductions and time-to-resolution improvements. The on-call benefit comes from completing the investigation before a human engages, so responders open a summarized conclusion instead of a raw alert. For a Talend context, that means a failed nightly load surfaces with a probable cause attached rather than a bare error count.

## 4. Developer support & automation (Bits AI Dev Agent)

---

The Dev Agent is a generative coding assistant that uses Datadog observability data to diagnose and fix code issues. It integrates with GitHub to open production-ready pull requests, iterates using CI logs and reviewer feedback, opens PRs as drafts and marks them ready when checks pass, and never auto-merges (you comment `@Datadog` to prompt updates). When Bits AI SRE finds a code-related root cause, it hands off to the Dev Agent automatically. It also fixes flaky tests via Test Optimization and generates fixes for SAST vulnerabilities in Code Security. The core Dev Agent is in Preview and requires requesting access; setup needs the `bits_dev_write` permission and the GitHub integration.

## 5. Security monitoring & triage (Bits AI Security Analyst)

---

Now generally available (Preview from June 2025), the Security Analyst autonomously triages Datadog Cloud SIEM signals, investigates potential threats in depth, and delivers reasoned recommendations without prompting. It draws on Datadog's internal security expertise, uses the MITRE ATT&CK framework to plan each investigation step, and pivots across indicators of compromise by querying data platform-wide. Datadog reports investigations that took hours can drop to as little as 30 seconds, with MTTR reductions of more than 90%. It fits a broader workflow: analysts escalate a signal into a Case auto-populated with metadata, then use Workflow Automation to run response steps such as notifying responders, updating tickets, or kicking off remediation runbooks.

## 6. Licensing & feature access

The Bits AI agents are **separately priced add-ons**, not bundled into an existing Datadog contract. Bits AI SRE uses investigation-based pricing, and only conclusive investigations are billable — inconclusive or incomplete ones are not charged. Because cost scales directly with alert volume, noisy alerting can consume budget quickly, so a PoC should scope auto-investigation to a few high-value monitors.

| Item                    | Detail                                                                                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Bits AI SRE (annual)    | ~\$500 / month per 20 conclusive investigations (~\$25 each)                                                                 |
| Bits AI SRE (monthly)   | ~\$600 / month per 20 investigations (~\$30 each)                                                                            |
| Bits AI SRE (on-demand) | Billed per individual investigation                                                                                          |
| Underlying products     | Log Management + APM/infra for SRE data; Cloud SIEM for Security Analyst; GitHub + <code>bits_dev_write</code> for Dev Agent |
| Platform tiers          | Free (up to 5 hosts), Pro (~\$15/host/mo), Enterprise (~\$23+/host/mo)                                                       |

Pricing and availability stages change frequently. Validate exact figures and entitlements against Datadog's pricing page and your account team before the PoC.

## 7. PoC approach & integrations

A sensible sequence for the Talend use case: (1) stand up the Agent on one Remote Engine / Job Server host and confirm parsed logs flow; (2) build one dashboard plus two or three monitors (job failure, error-rate spike, expected-job-absence); (3) wire those monitors to Slack and PagerDuty; (4) only then layer Bits AI SRE on top so investigations have real telemetry to reason over. The collaboration integrations are first-class entry points: connect the Slack workspace and reply to an alert with `@Datadog Investigate this alert`, or configure monitors to auto-launch an investigation on alert. ServiceNow and PagerDuty serve both as notification targets and, for ServiceNow, as a data source the agent can pull from. Keep PoC alert volume tight given per-investigation billing.

## 8. Automation & proactive monitoring (Bits AI + Watchdog)

Watchdog is the long-standing anomaly-detection layer (auto-detecting anomalies and outliers across APM and infrastructure without manual thresholds) and now feeds the proactive side of the Bits AI agents. The current direction connects detection to action, alongside new Proactive App Recommendations and APM Investigator capabilities. The practical loop is shown below: a monitor or Watchdog signal triggers an investigation, Bits AI SRE concludes and notifies, and the work branches to a Dev Agent code fix or a Workflow Automation remediation runbook.

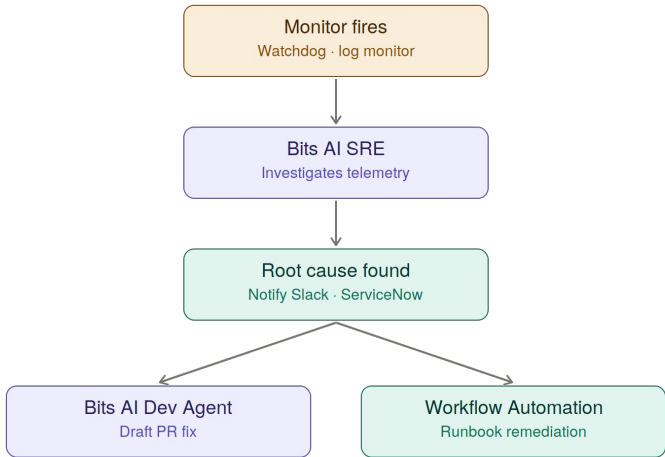


Figure 2 — The detect → investigate → remediate loop, including the branch to the Dev Agent (code fix) versus Workflow Automation (runbook).

## 9. Kubernetes monitoring setup

If Talend components (or the wider stack) run in Kubernetes, the setup is standardized. The Agent runs in-cluster and is configured via a DaemonSet spec, the Helm chart, or the Datadog Operator (Helm or Operator is recommended since it manages the DaemonSet, RBAC, and cluster-agent). Prerequisites are essentially a Datadog API key stored as a secret, Helm or the Operator installed, and RBAC for the Agent to read cluster metadata. With container log collection enabled, the Agent reads each container's stdout/stderr and auto-tags pod/namespace metadata, so containerized Talend jobs simply log to stdout and let Autodiscovery tag them — cleaner than file-tailing.

## 10. Unified observability dashboards

The end goal is a single pane correlating metrics, logs, and traces. Datadog dashboards blend all three signal types on shared widgets, and unified service tagging ( `env` , `service` , `version` ) is what makes correlation automatic — which is why the Agent config sets `service: talend-etl` consistently. A strong Talend dashboard combines job-level metrics from logs (success rate, runtime, throughput), error counts faceted by job name, JVM/host metrics from the same Agent, and links into the raw Log Explorer for drill-down. The same model extends across applications, databases, servers, network devices, and storage: consistent tagging lets widgets blend signals side by side, and Bits AI SRE can then reason across that correlated data during investigations.

## 11. Hands-on labs

These walk from a clean host to live alerting, then optionally to Kubernetes and the AI layer. Each lab ends with a validation step.

### LAB 1 Install the Agent and enable log collection

On the host running your Talend Job Server or Remote Engine:

```
DD_API_KEY=<YOUR_API_KEY> DD_SITE="datadoghq.com" \  
bash -c "$(curl -L https://install.datadoghq.com/scripts/install_script_agent7.sh)"
```

Enable logs in `/etc/datadog-agent/datadog.yaml`:

```
logs_enabled: true
```

Restart and validate (look for "Logs Agent => running"):

```
sudo systemctl restart datadog-agent  
sudo datadog-agent status
```

### LAB 2 Tail Talend's Log4j files

Create the custom source folder and config (Pattern A):

```
sudo mkdir -p /etc/datadog-agent/conf.d/talend.d  
sudo tee /etc/datadog-agent/conf.d/talend.d/conf.yaml > /dev/null <<'EOF'  
logs:  
  - type: file  
    path: /opt/talend/jobs/*/logs/*.log  
    source: talend  
    service: talend-etl  
    log_processing_rules:  
      - type: multi_line  
        name: collapse_java_stack_traces  
        pattern: \d{4}-\d{2}-\d{2}  
EOF  
sudo systemctl restart datadog-agent
```

The multi-line rule prevents each Java exception from splitting into many useless one-line events. If Talend's logs have restrictive permissions, adjust the file's group/read bits so the `dd-agent` user can tail it (Datadog advises against running the Agent as root). Validate in Logs → Log Explorer with the query `source:talend`.

### LAB 3 Make logs structured (so parsing is free)

Rather than write grok by hand, emit JSON from Talend. In Studio under File → Edit Project Properties → Project Settings → Log4j, switch the template to a JSON layout and push job context (job name, execution ID, status) into the MDC. Datadog auto-parses MDC key-value pairs into facets. If you cannot change the layout, build a grok parser instead: in Logs → Pipelines add a pipeline scoped to `source:talend`, add a Grok Parser to map `level`, `job_name`, `duration`, and set the status remapper so ERROR/FATAL color correctly.

### LAB 4 Log-based metric, monitor, and alert routing

Turn failures into a metric (Logs → Generate Metrics): define `talend.job.failures` from `source:talend status:error`, grouped by a `job_name` tag. Then create monitors (Monitors → New Monitor → Logs):

- Alert when `status:error service:talend-etl` count `> 0` over 5 minutes, grouped by `job_name`.
- Add a **log-absence** monitor that alerts when an expected nightly job logs no success line in its window — this catches schedules that silently never ran.
- In the notification message add `@slack-your-channel` and `@pagerduty-your-service` to route (configure those integrations first).

### LAB 5 Kubernetes (only if Talend runs in-cluster)

Install via Helm, which manages the DaemonSet and RBAC:

```
helm repo add datadog https://helm.datadoghq.com  
helm install datadog-agent \  
  --set datadog.apiKey=<YOUR_API_KEY> \  
  --set datadog.logs.enabled=true \  
  --set datadog.logs.containerCollectAll=true \  
datadog/datadog
```

With `containerCollectAll`, the Agent reads every container's stdout/stderr and auto-tags pod/namespace metadata. Validate with `kubectl get pods -l app=datadog-agent` and check the Explorer.

#### **LAB 6** Turn on AI investigation (optional, billed add-on)

Once monitors exist and Bits AI SRE is enabled, configure a monitor to launch an investigation automatically when it enters the alert state, or in Slack reply to the alert with `@Datadog Investigate this alert`. Scope this to a few high-value monitors during a PoC, since billing is per conclusive investigation.

## 12. Pre-flight checklist

---

- Confirm the real Talend log path — the glob `/opt/talend/jobs/*/logs/` varies by install and by Studio vs Remote Engine.
- Confirm the Log4j version is patched for Log4Shell before tailing production logs.
- Decide the deployment topology: self-managed Job Server, Talend Cloud Remote Engine, or containerized — this selects Pattern A, B, or the Kubernetes path.
- Ensure the `dd-agent` user can read the log files (group/read permissions, not root).
- Apply unified service tagging consistently so logs, metrics, and traces correlate.
- Validate exact Bits AI pricing and availability stages with your Datadog account team before committing.